

Worked Example: Silicon, kaldo vs. phonopy

May 9, 2026

*This document maps the substrate's abstract states and operations onto the concrete materializations produced by kaldo and phonopy/phono3py for crystalline silicon. It is the first contact between the architecture ([docs/symbolic_substrate.pdf](#)) and real physics. Equations are precise where kaldo and phonopy actually differ and sketchy where they do not. The exercise stops at *Dispersion*; scattering rates and the BTE solver are the next deliverable.*

1 Setup

System. Crystalline silicon, diamond structure, primitive cell with two atoms per unit cell.

Scope. Lattice thermal transport via the linearized Boltzmann transport equation, restricted to the harmonic approximation plus three-phonon scattering. Quantum statistics (f_{BE}). Quasi-harmonic Green–Kubo and four-phonon scattering deferred to later phases.

Reference codes.

- **kaldo** (Python). High-level API: `ForceConstants`, `Phonons`, `Conductivity`. Lazy evaluation; intermediate states exposed as attributes.
- **phonopy** + **phono3py** (Python+C). CLI-driven workflow, intermediate quantities in HDF5 files. Phonopy handles harmonic; phono3py handles anharmonic transport.

Conventions. Notation follows kaldo where it is unambiguous. Index conventions are not fixed at this stage; they become precise when adapters land.

Notation. V is the Born–Oppenheimer potential; $\{\mathbf{R}_i\}$ are atomic positions; u_i are displacements from equilibrium; M_α are atomic masses; $\mathbf{q}$ is a wavevector; ν is a band index.

2 Initial state: Potential

Abstract

The initial state is the Born–Oppenheimer potential energy surface,

$$V(\{\mathbf{R}_i\}) = V_0 + \sum_{n \geq 2} \frac{1}{n!} \sum_{i_1, \dots, i_n} \Phi_{i_1 \dots i_n}^{(n)} u_{i_1} \cdots u_{i_n}, \quad (1)$$

treated as a continuum mathematical object (an analytic function of the atomic positions in a neighborhood of equilibrium).

Substrate type: `POTENTIAL` (parameter state, empty provenance).

Materialization: the externally-supplied potential. In Phase 1 the upstream is stubbed:

the materialization is an opaque label identifying the choice of potential, not a modeled object.

Materialization in kaldo

kaldo does not represent the potential as a first-class object. The user supplies the potential by handing kaldo a *folder of pre-computed force constants* (see next section); the potential itself is implicit, accessed only through its second and third derivatives.

For the Tersoff path (`kaldo-examples/empirical_potentials/silicon_clathrate_Tersoff_LAMMPS`), the potential is realized as a LAMMPS pair-style declaration in `in.Si46`:

```
pair_style      tersoff
pair_coeff      * * forcefields/Si.tersoff Si
```

The materialization label is therefore `tersoff(Si.tersoff)`. The potential is callable (LAMMPS evaluates forces on demand) but not symbolically inspectable.

Materialization in phonopy / phono3py

Phonopy/phono3py likewise does not represent the potential as a first-class object. The user supplies displacements (generated by `phonopy -d` or `phono3py -d`) and externally runs a force calculator (VASP, QE, LAMMPS) on the displaced supercells. The potential is implicit in whichever code computed those forces.

For the canonical phono3py silicon example (`phono3py/example/Si-PBE`), the potential is QE/PBE with norm-conserving pseudopotentials. The materialization label is `dft(QE, PBE, Si.UPF, ecut=...)`. As with kaldo, the potential is callable through the external code but not symbolically inspectable.

Architectural note. Both adapters expose `Potential` as an opaque label upstream of the first computed state. This validates Phase 1’s stubbed-upstream choice. Neither code lets us extract the Taylor coefficients of V symbolically; both require numerical sampling. The real architectural strain shows up at the next state.

3 ForceConstants[order=2]

Abstract

The harmonic force constants are the second derivative of the potential at equilibrium,

$$\Phi_{i\alpha,j\beta}^{(2)} = \left. \frac{\partial^2 V}{\partial u_{i\alpha} \partial u_{j\beta}} \right|_0, \quad (2)$$

indexed by atom pairs (i,j) and Cartesian components (α,β) , evaluated on a finite supercell.

Substrate type: `FORCE_CONSTANTS`, with type-level parameter `order=2`.

Provenance (Phase 1, stubbed): a single operation `extract_derivative_and_truncate(Potential, order=2, method=M)` producing this state. The parameter $M \in \{\text{finite_difference_lammps}, \text{dfpt}, \text{finite_difference_qe}, \text{autodiff}, \dots\}$ is part of the operation’s identity; different M produce different states (same type, different provenance).

Approximation error: truncation of the Taylor expansion at order 2 introduces $O(u^3)$ error in the energy and $O(u^2)$ error in derived spectral quantities; these errors attach to the operation, not to this state.

Discretization (lives on the materialization): supercell $N_1 \times N_2 \times N_3$, finite-difference displacement step δ (when M is a finite-difference method).

Materialization in kaldo

kaldo loads pre-computed force constants from a folder:

```
forceconstants = ForceConstants.from_folder(
    folder='fc_Si46',
    supercell=(3, 3, 3),
    format='lammps',
)
```

Format-specific files (Tersoff path): `Dyn.form` (binary, LAMMPS PHONON output) plus `replicated_atoms.xyz`.

Internal storage: `forceconstants.second` is a `SecondOrder` object wrapping a numpy array of shape $(n_{\text{rep atoms}}, 3, n_{\text{rep atoms}}, 3)$.

Underlying method (M): for the Tersoff example, $M = \text{finite_difference_lammps}$. LAMMPS's `dynamical_matrix` command computes $\Phi^{(2)}$ by central-differencing forces with displacement $\delta = 10^{-5}$ Å (eskm 0.00001 in `in.Si46`).

Materialization in phonopy / phono3py

phonopy generates displacements, the user runs an external force calculator, and phonopy assembles the result into `fc2.hdf5`:

```
phonopy -d --dim 2 2 2 -c POSCAR-unitcell --pa auto    # generate displacements
# (run VASP/QE/LAMMPS on each disp-XXXXX/POSCAR)
phonopy -f disp-{00001..NNNN}/vasprun.xml             # collect forces
phonopy --writefc                                     # produce fc2.hdf5
```

File: `fc2.hdf5` (HDF5 with named datasets).

Internal storage: numpy array of shape $(n_{\text{atoms super}}, n_{\text{atoms super}}, 3, 3)$ accessible as `phonopy.force_constants`.

Underlying method (M): typically `finite_difference_{ext_code}` with phonopy's symmetry-reduced displacement set; the displacement magnitude defaults to 0.01 Å (overridable via `--amplitude`).

Architectural strain (worth flagging). The abstract framing “ $\Phi^{(2)} = \partial^2 V / \partial u^2$ ” suggests symbolic differentiation, but neither code performs it. Both numerically sample V at displaced configurations and apply finite-difference rules. This is the Tersoff/Taylor mismatch flagged in the planning discussion.

The substrate handles this honestly because *the method of derivative extraction is a parameter on the operation's identity*. The same abstract `ForceConstants[order=2]` type admits multiple operations:

- `extract_derivative_and_truncate(method=finite_difference_lammps,  $\delta=10^{-5}$ )`
- `extract_derivative_and_truncate(method=finite_difference_qe,  $\delta=10^{-2}$ )`

- `extract_derivative_and_truncate(method=dfpt,  $\epsilon_{\text{conv}}$ =...)`
- `extract_derivative_and_truncate(method=autodiff)`

Each is a distinct operation (parameterized identity, §3.5.1 of the design doc); each produces a state with the same physics type but different provenance. “Same physics object, computed differently” becomes a structural distinction in the substrate, not a metadata tag.

4 ForceConstants[order=3]

Abstract

The cubic force constants are the third derivative,

$$\Phi_{i\alpha, j\beta, k\gamma}^{(3)} = \left. \frac{\partial^3 V}{\partial u_{i\alpha} \partial u_{j\beta} \partial u_{k\gamma}} \right|_0. \quad (3)$$

Substrate type: `FORCE_CONSTANTS`, type-level parameter `order=3`.

Provenance: `extract_derivative_and_truncate(Potential, order=3, method= $M'$ )`. The method M' for third-order is typically *different* from the second-order method: DFPT does not extend cleanly to third order, so M' is usually finite difference even when M is DFPT.

Approximation error: truncation at order 3 introduces $O(u^4)$ error (four-phonon and higher scattering, anharmonic renormalization, etc.).

Discretization: a third-order supercell $N'_1 \times N'_2 \times N'_3$, displacement step δ' . Crucially, $N' \neq N$ in general — the third-order supercell is typically smaller than the second-order one because the cost scales worse.

Materialization in kaldo

```
forceconstants = ForceConstants.from_folder(
    folder='fc_DFT',
    supercell=(8, 8, 8),          # 2nd-order supercell
    third_supercell=(3, 3, 3),    # 3rd-order supercell
    format='qe-sheng',
)
```

Format-specific files: `FORCE_CONSTANTS_3RD` (Sheng-style plain text) for QE path; `THIRD` (binary) for the LAMMPS path.

Internal storage: `forceconstants.third` is a `ThirdOrder` object; internal layout uses the `thirdorder.py` sparse convention by default.

Underlying method (M'): finite-difference of forces, supercell typically $3 \times 3 \times 3$ for affordability.

Materialization in phonopy / phono3py

phono3py mirrors the phonopy workflow at third order:

```
phono3py -d --dim 2 2 2 -c POSCAR-unitcell          # 3rd-order displacements
phono3py --cf3 disp-{00001..NNNNN}/vasprun.xml      # collect forces
phono3py-load                                         # produce fc3.hdf5
```

File: `fc3.hdf5`.

Internal storage: numpy array of shape $(n_{\text{atoms super}})^3 \times (3, 3, 3)$ accessible via

Phono3py.fc3 (compact form available too).

Underlying method (M'): finite-difference, displacement amplitude default 0.03 Å for third order (larger than the second-order default to improve signal relative to higher-order noise).

Note on supercell asymmetry. Both codes accept different supercells for second and third order. In the substrate, this means `ForceConstants[order=2]` and `ForceConstants[order=3]` are independent derived states with independent discretizations on their materializations. They share the same `Potential` as an upstream parameter but are otherwise unrelated by the abstract DAG. Good test of the type-level `order` parameter doing real work.

5 Correction operation: `enforce_acoustic_sum_rule`

Abstract

A correction operation that takes a force-constant tensor and returns a corrected one with exact translational invariance enforced:

$$\tilde{\Phi}_{i\alpha,j\beta}^{(2)}(R) \text{ such that } \sum_R \tilde{\Phi}_{i\alpha,j\beta}^{(2)}(R) = 0 \quad \forall i, \alpha, \beta. \quad (4)$$

This is Newton’s third law in real space: the crystal’s energy is invariant under uniform translation, so the rows and columns of the FC tensor must each sum to zero. Numerical FCs from finite-difference (or, less severely, from DFPT) violate this approximately; uncorrected, the violation appears as small spurious imaginary frequencies near $\mathbf{q} = 0$.

Substrate type signature: `enforce_acoustic_sum_rule : FC[order= $n$ ]  $\rightarrow$  FC[order= $n$ ].`

Operation parameters:

`variant` $\in$ {`simple_subtract`, `iterative_lstsq`, `born_huang`}.

Approximation error: $\epsilon_{\mathcal{O}} = 0$ in the abstract world (the operation is exact on the abstract object). The correction’s effect on the materialization is bounded by $|\sum_R \Phi(R)|$ before enforcement, which is itself a discretization-error indicator for the input FC tensor.

Provenance: the input and output have the same physics type `FORCE_CONSTANTS` with the same `order`, but the output’s provenance is the input’s extended by this operation. They are formally distinct states (Decision 5) and downstream operations consume the ASR-enforced output, not the raw input.

This is the first “correction operation” in the worked example. Its shape—input and output share the same physics type, distinguished only by an additional provenance entry—is the template for all corrections (LO–TO splitting on `DynamicalMatrix`, permutation symmetrization on `FORCE_CONSTANTS[order=3]`, etc.).

Materialization in `kaldo`

`kaldo` enforces ASR at FC load via a flag on `from_folder`:

```
forceconstants = ForceConstants.from_folder(
    ...
    is_acoustic_sum=True,      # <-- triggers ASR
)
```

Internally, `kaldo` applies a simple residual-subtraction variant: for each (i, α, j, β) , the residual $\sum_R \Phi^{(2)}(R)$ is divided over the on-site terms ($R = 0$) so that the new sum is zero exactly.

Variant: `simple_subtract`.

Timing: at FC load (before any q-space computation).

Default: *off*. The user must opt in.

Materialization in `phonopy` / `phono3py`

`phonopy` enforces ASR via the `--fc-symmetry` flag, applied during `phonopy --writefc`:

```
phonopy --fc-symmetry --writefc
```

The implementation iteratively projects out residual translations and applies permutation symmetry $(i, \alpha) \leftrightarrow (j, \beta)$, looping until both invariances hold simultaneously within a tolerance.

Variant: `iterative_lstsq` (combined with permutation symmetrization, which we treat as a separate operation).

Timing: at FC write (before `fc2.hdf5` is produced).

Default: on (when `--writefc` is invoked) for current versions; off in older `phonopy`.

Cross-code structural observation. `kaldo` and `phonopy` apply ASR *at different points in their internal pipelines* (load vs. write) and use *different variants* (simple subtraction vs. iterative). In the substrate, both adapters declare the same abstract operation `enforce_acoustic_sum_rule`, with the variant carried as an operation parameter. The internal-pipeline timing is invisible to the substrate; what the substrate sees is that both adapters' FC materializations downstream of `enforce_asr` can be aligned. The variant parameter makes the methodological difference visible without forcing the substrate to know where, internally, the codes apply it.

Note on order=3. Acoustic-sum-rule-like enforcement on third-order FCs (more properly, permutation symmetrization plus the order=3 acoustic invariance) is a separate correction operation in the substrate, with the same shape: `enforce_acoustic_sum_rule : FC[order=3] → FC[order=3]`. We do not detail it here because `kaldo` applies it implicitly via `thirdorder.py`'s convention, while `phono3py` applies it during `fc3.hdf5` production. The cross-code variant comparison follows the same pattern as for order=2.

6 DynamicalMatrix

Abstract

The dynamical matrix is the mass-weighted Fourier transform of the harmonic force constants,

$$D_{\alpha\beta}^{ij}(\mathbf{q}) = \frac{1}{\sqrt{M_i M_j}} \sum_{\mathbf{R}} \Phi_{i\alpha, j\beta}^{(2)}(\mathbf{R}) e^{i\mathbf{q} \cdot \mathbf{R}}. \quad (5)$$

Substrate type: `DYNAMICAL_MATRIX`.

Operation: `compute_dynamical_matrix(ForceConstants[order=2])`, structural operation, no approximation, $\epsilon_{\mathcal{O}} = 0$. The expected input is the ASR-enforced `ForceConstants[order=2]` produced by `enforce_acoustic_sum_rule`; consuming the raw FC tensor is structurally legal but produces a dynamical matrix with non-zero

acoustic-mode frequencies at $\mathbf{q} = 0$, which downstream operations may flag.

Discretization: q-mesh $K_1 \times K_2 \times K_3$ on which $D(\mathbf{q})$ is sampled.

Sketchy on purpose. kaldo and phonopy do not disagree on what the dynamical matrix is. The only divergence at this stage is whether LO–TO splitting has been added (relevant for polar materials, irrelevant for silicon). A separate operation `apply_lo_to_correction` would handle this where it matters; we omit it here for silicon.

Materialization in kaldo

Computed lazily on access:

```
phonons = Phonons(forceconstants=fc, kpts=[14, 14, 14], temperature=300, ...)
phonons.dynmat                                # triggers computation
```

Internal Cython kernel: `kaldo/observables/harmonic_with_q.py`.

Materialization in phonopy / phono3py

```
phonopy.run_band_structure(paths=...)          # or
phonopy.dynamical_matrix.set_dynamical_matrix(q)
phonopy.dynamical_matrix.dynamical_matrix      # 2x2 block per atom pair
```

Implementation: `phonopy/harmonic/dynamical_matrix.py`.

7 Dispersion

Abstract

The dispersion is the eigenstructure of the dynamical matrix,

$$D(\mathbf{q}) \mathbf{e}_{\mathbf{q}\nu} = \omega_{\mathbf{q}\nu}^2 \mathbf{e}_{\mathbf{q}\nu}, \quad \omega_{\mathbf{q}\nu} \geq 0, \quad \mathbf{e} \cdot \mathbf{e}^* = 1. \quad (6)$$

The state carries the mode frequencies $\omega_{\mathbf{q}\nu}$, the polarization vectors $\mathbf{e}_{\mathbf{q}\nu}$, and the group velocities $\mathbf{v}_{\mathbf{q}\nu} = \nabla_{\mathbf{q}} \omega_{\mathbf{q}\nu}$ (derived from Hellmann–Feynman or finite-differencing on the q-mesh).

Substrate type: DISPERSION.

Operation: `compute_dispersion(DynamicalMatrix)`, structural operation, $\epsilon_0 = 0$.

Discretization: same q-mesh as `DynamicalMatrix`.

Sketchy on purpose. Hermitian eigendecomposition is unambiguous. Group-velocity extraction has a small implementation choice (analytic via Hellmann–Feynman vs. finite-difference on the mesh) but the difference is negligible for silicon at standard mesh densities.

Materialization in kaldo

```
phonons.frequency          # shape (n_q, n_modes), units THz
phonons.eigenvectors       # shape (n_q, n_modes, n_modes)
phonons.velocity           # group velocities, shape (n_q, n_modes, 3)
```

Frequencies are stored in THz internally. Group velocities computed via Hellmann–Feynman inside `harmonic_with_q.py`.

Materialization in phonopy / phono3py

```
phonopy.set_band_structure(paths=...)
phonopy.get_band_structure_dict()      # returns {'frequencies', 'eigenvectors', 'group_v
```

Frequencies in THz. Group velocities via the `group_velocity` module (Hellmann–Feynman).

8 What we learned

The first contact between architecture and physics surfaced four observations.

1. **The upstream stub is honest.** Both `kaldo` and `phonopy` treat the potential as an opaque label. The user supplies it externally (Tersoff parameters, DFT functional, ML weights) and neither code symbolically inspects V . Phase 1’s stubbed-upstream choice matches both adapters exactly: `Potential` is a parameter state with an opaque-label materialization, and the upstream operations are recorded as provenance only.
2. **The Tersoff/Taylor mismatch is not a bug; it’s a feature of the parameterized operation identity.** The same abstract derivative-extraction operation admits multiple methods (finite difference, DFPT, autodiff). Each method is a distinct operation identity. Materializations of the same `ForceConstants[order=n]` type produced by different methods carry different provenance and are formally distinct — which is exactly what we want for cross-method comparison.
3. **Second- and third-order force constants are independent derived states.** Different supercells, different methods, independent discretizations. The type-level `order` parameter is doing real work in the substrate; `FORCE_CONSTANTS` with no `order` parameter would be ambiguous.
4. **Structural operations (Fourier, eigendecomposition) are unambiguous.** `kaldo` and `phonopy` implement them in different files, in different languages, with different internal data layouts, but both produce the same abstract state. These operations are good places to validate the materialization-functor alignment machinery before introducing operations where the codes actually disagree.
5. **Correction operations have a uniform shape.** `enforce_acoustic_sum_rule` (and the LO–TO splitting, permutation symmetrization, and Born–Huang invariance operations that will follow) all share the pattern $\tau \rightarrow \tau$: same input and output type, distinguished only by an additional provenance entry on the output. By Decision 5 (identity = type + provenance), input and output are formally distinct states despite sharing a type. This pattern lets us factor cross-code disagreements that live in correction *variants* and *timing*, not in the operation’s structural type signature.

No architectural revisions required at this stage. The substrate’s commitments (Principles 1, 3, 5, 6, 7, 9) survive the contact with this fragment of the workflow. The discriminative test — whether the substrate exposes the broadening, scattering-process, and BTE-solver choices where `kaldo` and `phono3py` actually diverge — comes in the next deliverable when we extend the worked example through `ScatteringRates` and `BoltzmannSolution`.